Toolchain, and How a Program Becomes a Program

Keep this. You will want the four-stage pipeline and key terms again in later weeks — this course does not repeat toolchain explanations once the semester's projects start. (Compiler warnings, the command reference, and the VS Code/Cline tour are in the Lecture 2 handout.)

1. The shape of a C++ program

Before anything else: recognize these pieces on sight. You do not need to know how any of them work yet.

Piece	What it is
#include "..." / <...>	Pulls in code someone else wrote. Mechanically explained later this lecture.
int main() { }	Every C++ program has exactly one. Execution starts here.
; (semicolon)	Ends a statement — one complete instruction.
{ } (braces)	Mark a block: a group of statements that belong together.
// text	A comment. The compiler ignores it entirely — for humans only.
std::cout << ...;	Prints text to the screen. Properly taught week 3 — for now, just recognize it.

2. The four-stage pipeline

A single .cpp file becomes a running program by passing through four independent stages. Each stage has its own job, its own input file type, and its own kind of error.

```
your_file.cpp
      |  preprocessor   (textual: expands #include, #define)
      v
your_file.i           <- "translation unit": one .cpp + everything it #included
      |  compiler      (syntax + type checking happen HERE)
      v
your_file.s           <- assembly (you will never write this by hand)
      |  assembler
      v
your_file.o           <- machine code, but not yet runnable
      |  linker        (combines every .o the program needs)
      v
game                  <- the actual, runnable executable
```

Which stage produces which error

You see...	It happened at...	Meaning
"expected ';' before..."	compile	Syntax error — the compiler could not parse your code.
"'x' was not declared..."	compile	You used a name with no matching declaration in scope.
"undefined reference to 'f()'"	link	Every .cpp compiled fine, but nothing anywhere defines f(). Check spelling, and that the .cpp defining it is in the build.
"multiple definition of 'f()'"	link	Two translation units both define the same function/variable — a One-Definition-Rule violation.

3. Key terms

Term	Definition
Translation unit	One .cpp file, plus every header it pulls in via #include, after the preprocessor has finished. This is what the compiler actually compiles — one at a time, with no knowledge of any other .cpp file.
One-Definition Rule (ODR) — taught week 3	Exactly one definition of any given function, variable, or class may exist across the whole linked program. Violating it is usually a linker error, occasionally silent undefined behavior.
Object file (.o)	Machine code for one translation unit. Not runnable by itself — it can contain calls to functions the linker has not resolved yet.
Linker	Combines every .o file a program needs into one executable, resolving each call to the actual address of its definition.